

Hands-on Lab: Verifying Data Quality for a Data Warehouse



Estimated time needed: 30 minutes

Purpose of the Lab:

The primary purpose of this lab is to instruct participants on the process of conducting thorough data quality checks in a data warehousing environment. It focuses on using a Python-based framework within a PostgreSQL database to validate data integrity. Key areas of emphasis include identifying null values, duplicates, and invalid entries, as well as verifying data ranges. The lab aims to equip learners with the necessary skills to set up and utilize a testing framework for data validation, ensuring data accuracy and consistency.

Benefits of Learning the Lab:

Engaging in this lab offers several benefits, particularly in enhancing one's capabilities in data management and quality assurance. Learners will gain hands-on experience in implementing automated data quality checks, a skill crucial for maintaining the reliability of data in real-world applications. This proficiency is especially beneficial for professionals working with large datasets, as it ensures the integrity of data used for analysis and decision-making. Moreover, understanding these concepts is essential for anyone aspiring to specialize in data science, database administration, or any field that relies heavily on accurate and reliable data.

Objectives

In this lab, you will:

- Check Null values
- Check Duplicate values
- Check Min Max
- Check Invalid values
- Generate a report on data quality

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. Theia is an open source IDE (Integrated Development Environment), that can be run on desktop or on the cloud. To complete this lab, we will be using the Cloud IDE based on Theia running in a Docker container.

Important Notice about this lab environment

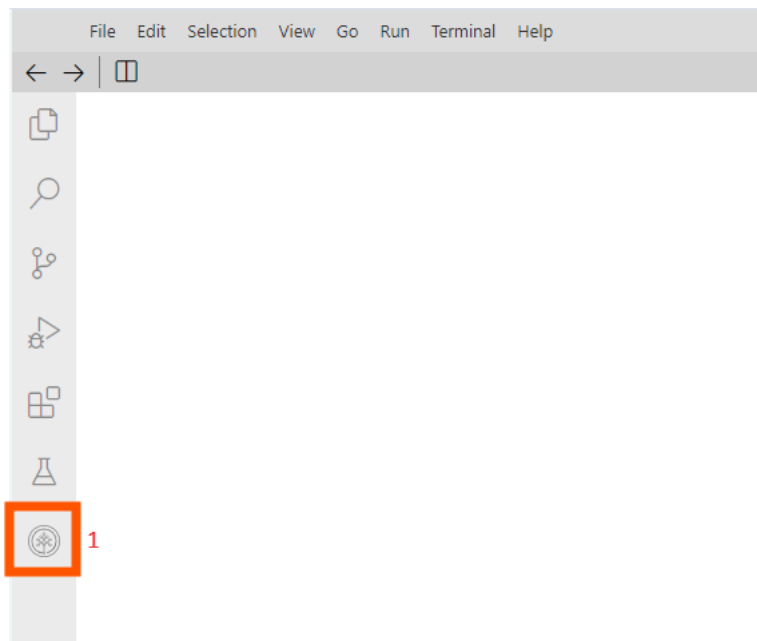
Please be aware that sessions for this lab environment are not persistent. A new environment is created for you every time you connect to this lab. Any data you may have saved in an earlier session will get lost. To avoid losing your data, please plan to complete these labs in a single session.

Exercise 1 - Getting the environment ready

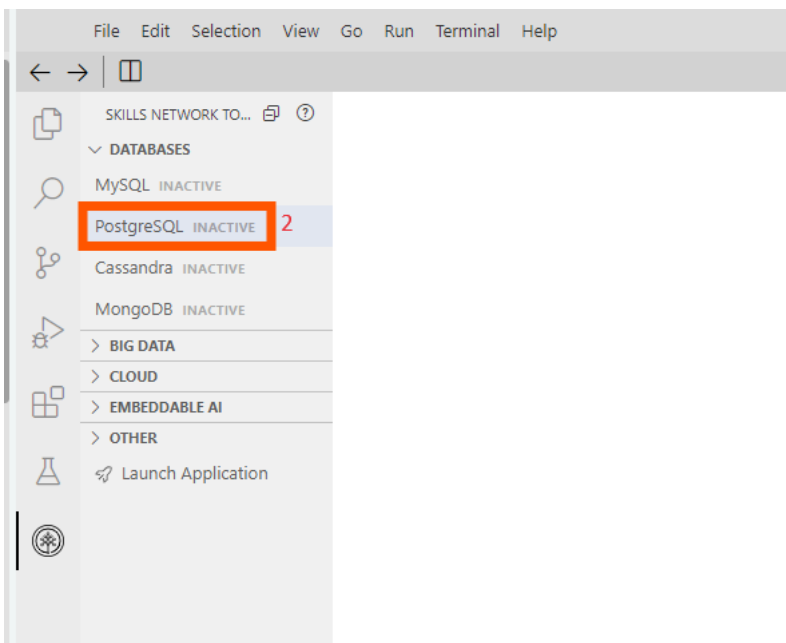
In this exercise, you will get the environment ready so that we can perform the data quality checks.

Step 1: Start the postgresql server.

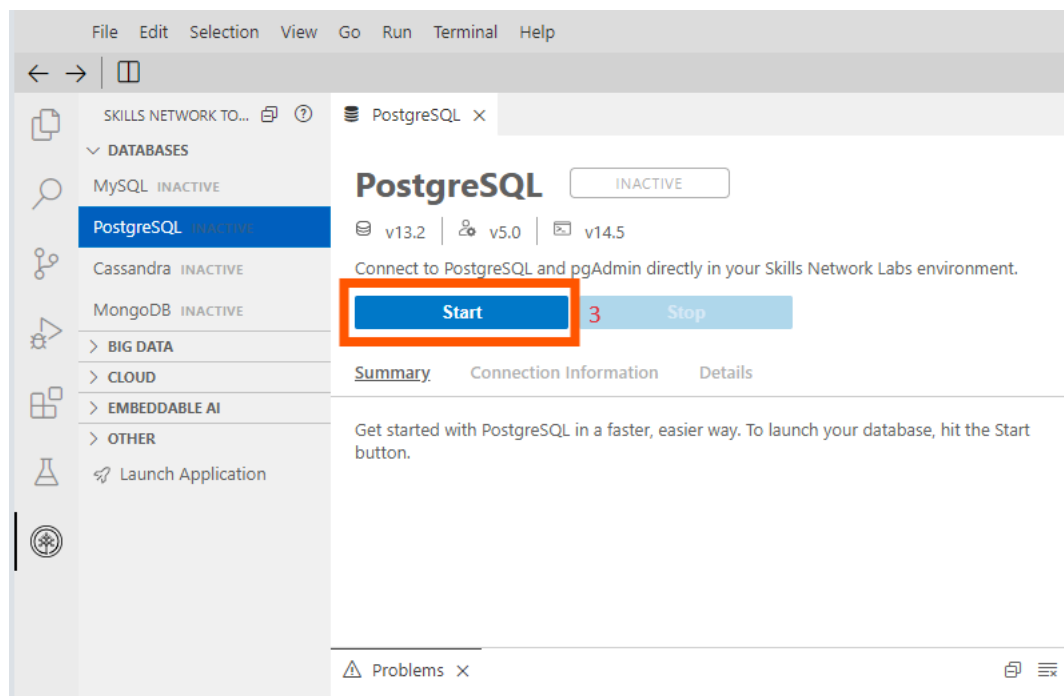
1. Click on **Skills Network toolbox**.



2. Select **PostGres** database.

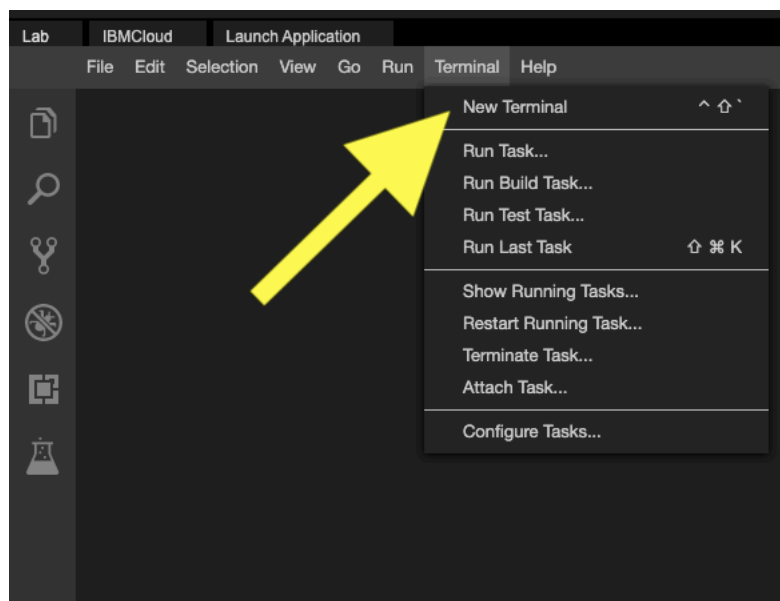


3. click on **Start** button.

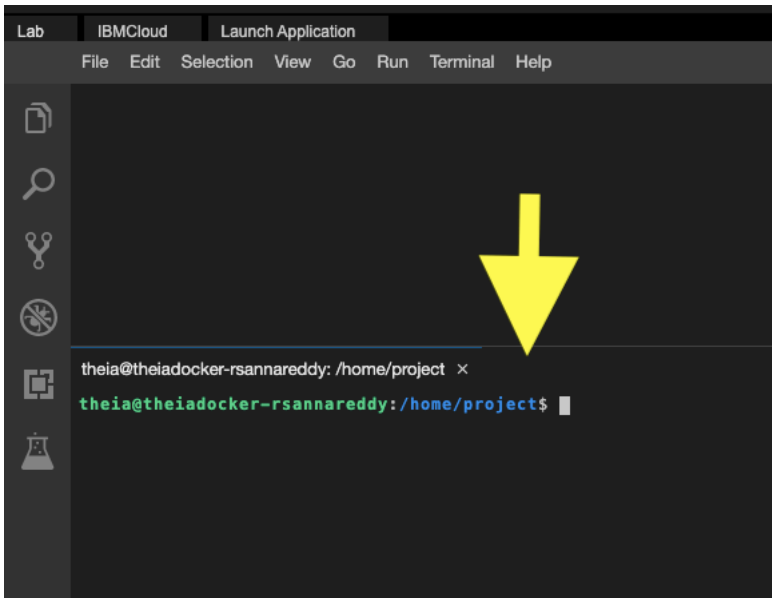


Step 2: Download the staging area setup script.

Open a new terminal, by clicking on the menu bar and selecting **Terminal->New Terminal**, as shown in the image below.



This will open a new terminal at the bottom of the screen.



Run the command below to download the staging area setup script.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/8ZukKar_boDbhNgMiwGAWg/setupstagingarea.sh
```

Step 3: Export your postgres password.

Run the below command in the terminal by replacing <your postgres password> with your postgres password that can be found under the connection information tab.

export PGPASSWORD=<your postgres password>

Step 4: Run the setup script.

Run the command below to execute the staging area setup script.

```
bash setupstagingarea.sh
```

When you see a message `Successfully setup the staging area` you are ready to perform data quality checks.

Exercise 2 - Getting the testing framework ready

You can perform most of the data quality checks by manually running sql queries on the data warehouse.

It is a good idea to automate these checks using custom programs or tools. Automation helps you to easily

- create new tests,
- run tests,
- and schedule tests.

We will be using a python based framework to run the data quality tests.

Step 1: Download the framework.

Run the commands below to download the framework

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DB0260EN-SkillsNetwork/labs/Verifying%20Data%20Quality%20for%20a%20Data%20Warehouse
```

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/HB0XK4MDrGwigMmVPmPoeQ/dbconnect.py
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DB0260EN-SkillsNetwork/labs/Verifying%20Data%20Quality%20for%20a%20Data%20
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/saTxV8y9Kt-e8Zxe29M0TA/generate-data-quality-report.py
ls
```

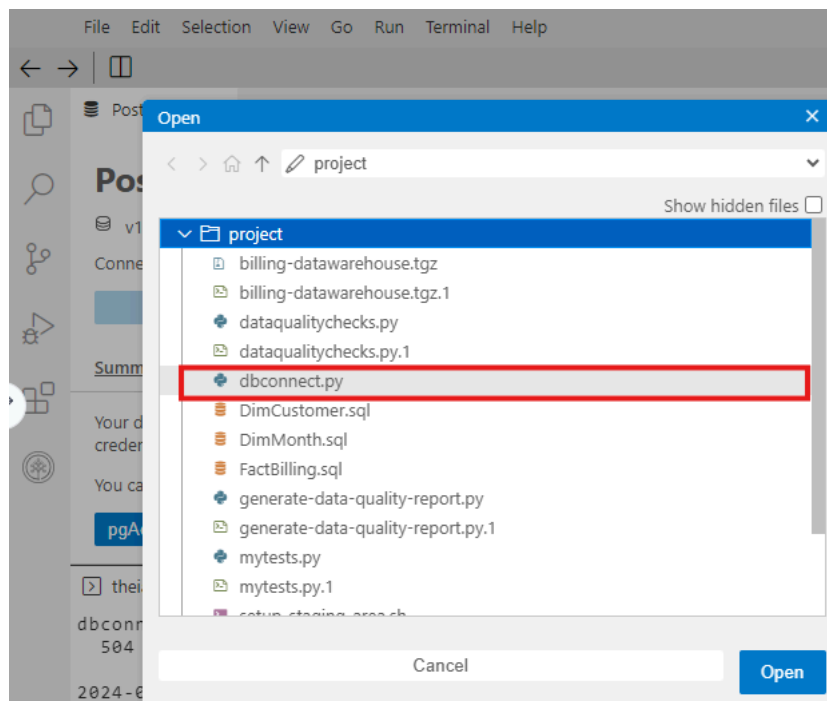
Step 2: Install the python driver for Postgresql.

Run the command below to install the python driver for Postgresql database

```
python3 -m pip install psycopg2
```

Step 3: Update Password in dbconnect.py

Go to File>Open. A window will appear; double-click on dbconnect.py to open it.



Once the file is open, in line 3, replace **<replace this with your postgres password>** with your **POSTGRES_PASSWORD** which can be found under the **Connection Information** tab when you scroll down.

Once done, save the file by selecting **File>Save**.

```

1 import os
2 import psycopg2
3 pgpassword = "<replace this with your postgres password>"
4 conn = None
5 try:
6     conn = psycopg2.connect(
7         user = "postgres",
8         password = pgpassword,
9         host = "postgres",
10        port = "5432",
11        database = "postgres")

```

PostgreSQL ACTIVE

v13.2 | v5.0 | v14.5

Connect to PostgreSQL and pgAdmin directly in your Skills Network Labs environment.

Create Delete

Summary **Connection Information** Details

POSTGRES_USERNAME: postgres

POSTGRES_HOST: 172.21.231.77

Repeat the above step 3 for generate-data-quality-report.py file as well. Update your password in line 15 of the script and save it.

Step 4: Test database connectivity.

Now we need to check

- if the Postgresql python driver is installed properly.
- if Postgresql server is up and running.
- if our micro framework can connect to the database.

The command below to check all the above cases.

```
python3 dbconnect.py
```

If all goes well, you should a message Successfully connected to database.

The command also disconnects from the server with a message Connection closed.

Exercise 3 - Create a sample data quality report

Run the command below to install pandas.

```
python3 -m pip install pandas tabulate
```

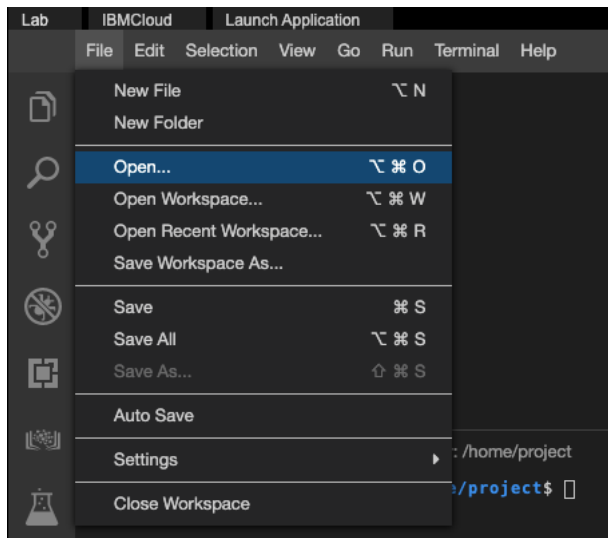
Run the command below to generate a sample data quality report.

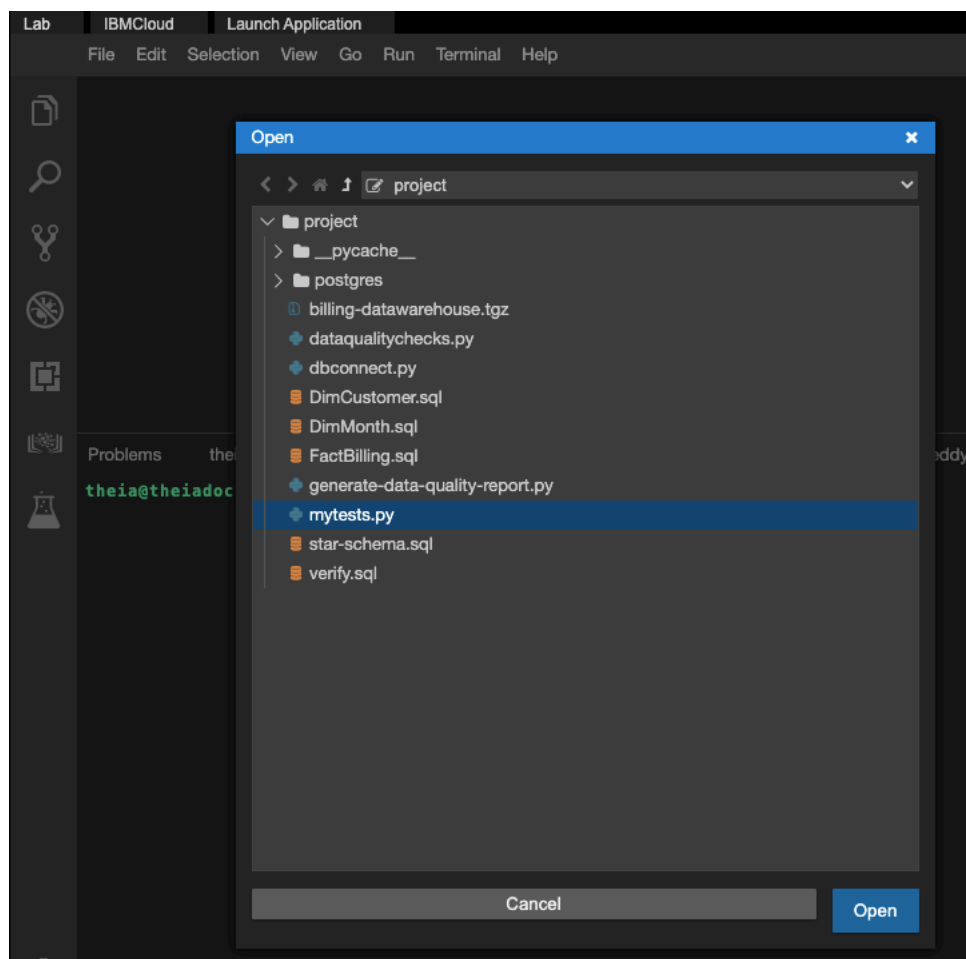
```
python3 generate-data-quality-report.py
```

You should see a list of tests that were run and their status.

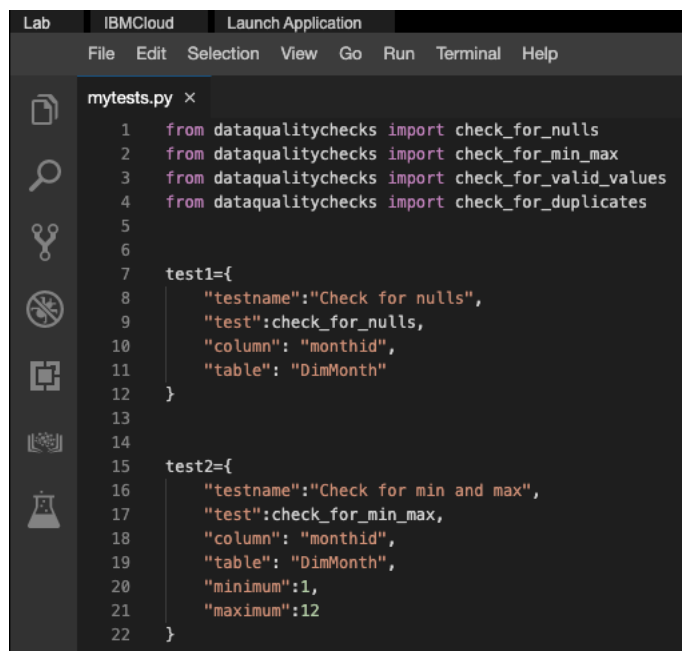
Exercise 4 - Explore the data quality tests

Open the file `mytests.py` in the editor by using the steps below.





You should now see the file opened in the editor.



The file `mytests.py` contains all the data quality tests.

It provides a quick and easy way to author and run new data quality tests.

The testing framework provides the following tests:

- `check_for_nulls` - this test will check for nulls in a column
- `check_for_min_max` - this test will check if the values in a column are with a range of min and max values
- `check_for_valid_values` - this test will check for any invalid values in a column
- `check_for_duplicates` - this test will check for duplicates in a column

Each test can be authored by mentioning a minimum of 4 parameters.

- testname - The human readable name of the test for reporting purposes
- test - The actual test name that the testing micro framework provides
- table - The table name on which the test is to be performed
- column - The table name on which the test is to be performed

Exercise 5 - Check for nulls

Let us now see what a `check_for_nulls` test looks like.

Here is a sample `check_for_nulls` test:

```
test1={
  "testname":"Check for nulls",
  "test":check_for_nulls,
  "column": "monthid",
  "table": "DimMonth"
}
```

All tests must be named as `test` following by a unique number to identify the test.

- Give an easy to understand description for `testname`
- mention `check_for_nulls` for `test`
- mention the column name on which you wish to check for nulls
- mention the table name where this column exists

Let us now create a new `check_for_nulls` test and run it.

The test below checks if there are any null values in the column `year` in the table `DimMonth`.

The test fails if nulls exist.

Copy and paste the code below at the end of `mytests.py` file.

```
test5={
  "testname":"Check for nulls",
  "test":check_for_nulls,
  "column": "year",
  "table": "DimMonth"
}
```

Save the file using Menu -> File -> Save

Run the command below to generate the new data quality report.

```
python3 generate-data-quality-report.py
```

Exercise 6 - Check for min max range

Let us now see what a `check_for_min_max` test looks like.

Here is a sample `check_for_min_max` test

```
test2={
  "testname":"Check for min and max",
  "test":check_for_min_max,
  "column": "monthid",
  "table": "DimMonth",
  "minimum":1,
  "maximum":12
}
```

In addition to the usual fields, you have two more fields here.

- minimum is the lowest valid value for this column. (Example 1 in case of month number)
- maximum is the highest valid value for this column. (Example 12 in case of month number)

Let us now create a new `check_for_min_max` test and run it.

The test below checks for minimum of 1 and maximum of 4 in the column `quarter` in the table `DimMonth`.

The test fails if there any values less than minimum or more than maximum.

Copy and paste the code below at the end of `mytests.py` file.

```
test6={
  "testname":"Check for min and max",
  "test":check_for_min_max,
  "column": "quarter",
  "table": "DimMonth",
  "minimum":1,
  "maximum":4
}
```

Save the file using Menu -> File -> Save

Run the command below to generate the new data quality report.

```
python3 generate-data-quality-report.py
```

Exercise 7 - Check for any invalid entries

Let us now see what a `check_for_valid_values` test looks like.

Here is a sample `check_for_valid_values` test:

```
test3={
  "testname":"Check for valid values",
  "test":check_for_valid_values,
  "column": "category",
  "table": "DimCustomer",
  "valid_values":{'Individual','Company'}
}
```

In addition to the usual fields, you have an additional field here.

- use the field `valid_values` to mention what are the valid values for this column.

Let us now create a new `check_for_valid_values` test and run it.

The test below checks for valid values in the column `quartername` in the table `DimMonth`.

The valid values are Q1,Q2,Q3,Q4

The test fails if there any values less than minimum or more than maximum.

Copy and paste the code below at the end of `mytests.py` file.

```
test7={
    "testname":"Check for valid values",
    "test":check_for_valid_values,
    "column": "quartername",
    "table": "DimMonth",
    "valid_values":{'Q1','Q2','Q3','Q4'}
}
```

Save the file using Menu -> File -> Save

Run the command below to generate the new data quality report.

```
python3 generate-data-quality-report.py
```

Exercise 8 - Check for duplicate entries

Let us now see what a `check_for_duplicates` test looks like.

Here is a sample `check_for_duplicates` test

```
test4={
    "testname":"Check for duplicates",
    "test":check_for_duplicates,
    "column": "monthid",
    "table": "DimMonth"
}
```

Let us now create a new `check_for_duplicates` test and run it.

The test below checks for any duplicate values in the column `customerid` in the table `DimCustomer`.

The test fails if duplicates exist.

Copy and paste the code below at the end of `mytests.py` file.

```
test8={
    "testname":"Check for duplicates",
```

```
"test":check_for_duplicates,  
"column": "customerid",  
"table": "DimCustomer"  
}
```

Save the file using Menu -> File -> Save

Run the command below to generate the new data quality report.

```
python3 generate-data-quality-report.py
```

Practice exercises

1. Problem:

Create a check_for_nulls test on column billedamount in the table FactBilling

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

2. Problem:

Create a check_for_duplicates test on column billid in the table FactBilling

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

3. Problem:

Create a check_for_valid_values test on column quarter in the table DimMonth. The valid values are 1, 2, 3, 4

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

Congratulations!! You have successfully finished this lab.

Authors

Ramesh Sannareddy

Other Contributors

Rav Ahuja

© IBM Corporation. All rights reserved.